

Motivation and Introduction to RE

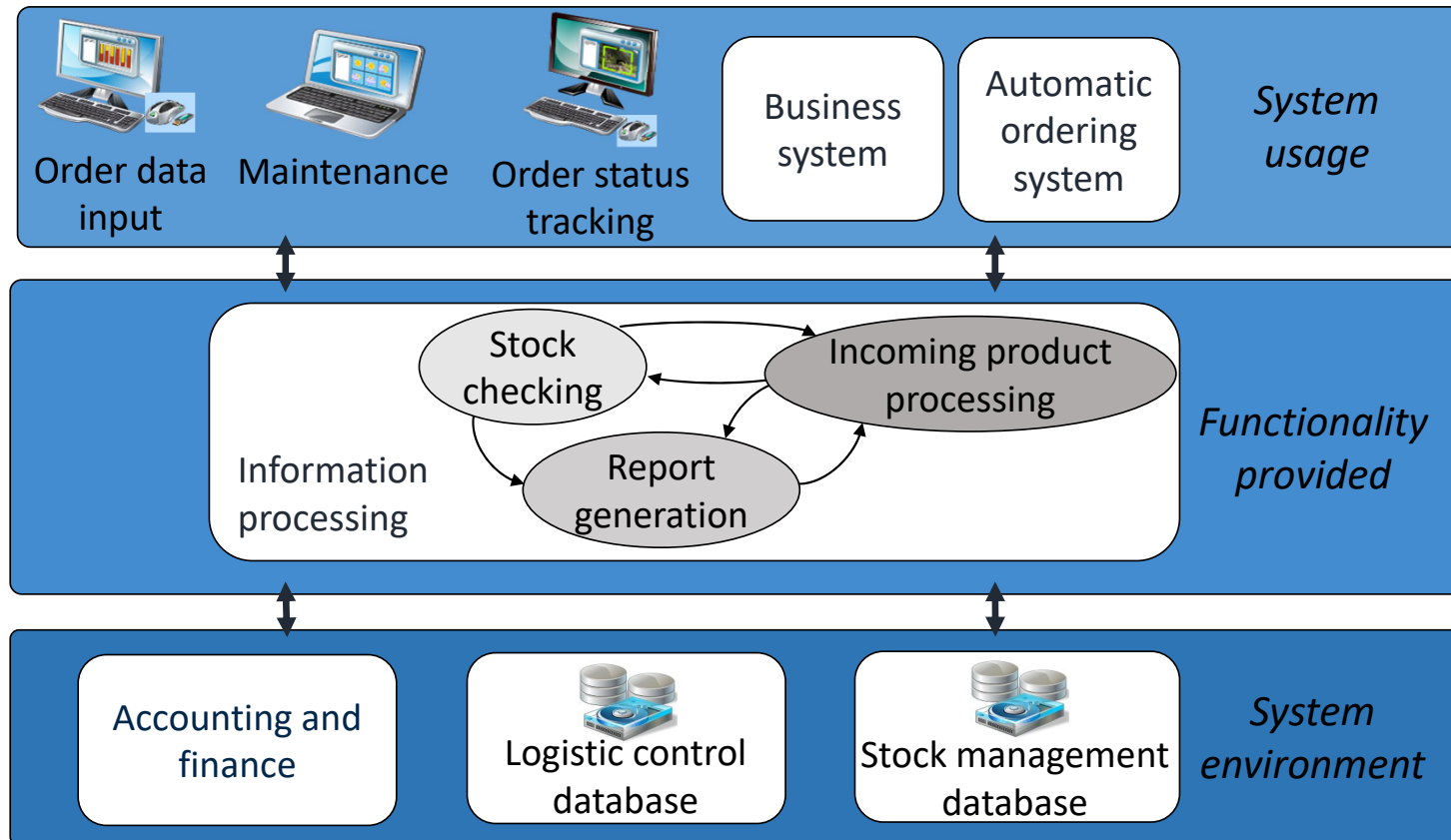
Bonita Sharif
School of Computing
University of Nebraska - Lincoln

Four Types of Systems

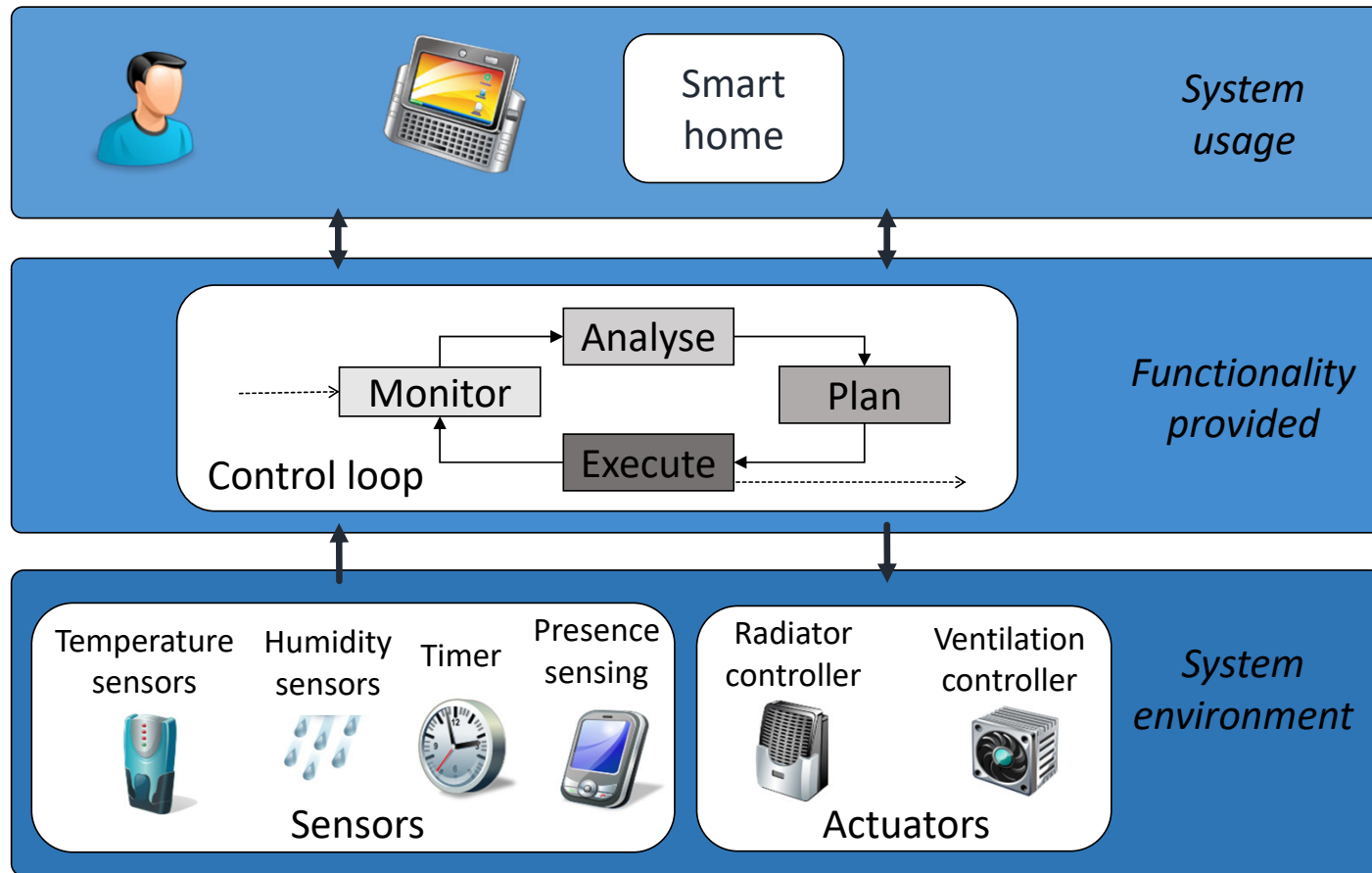
- Information systems
 - Collect, process, store, analyse, and disseminate information, e.g., for business processes
 - Provide functionality via sophisticated interfaces; large amount of (complex) data
- Embedded systems
 - Implement a control loop to control processes in the environments
 - Tight integration of hardware, electronic parts and software; real-time requirements
- Software-intensive systems
 - Combine characteristics of embedded and information systems; control loop and information processing
 - Provide complex functions, including innovative functionality and quality
- Adaptive systems
 - Additional adaptation logic (MAPE-K loop) to monitor the context, analyse data and context information to identify, plan and executed adaptations during run-time

Information Systems

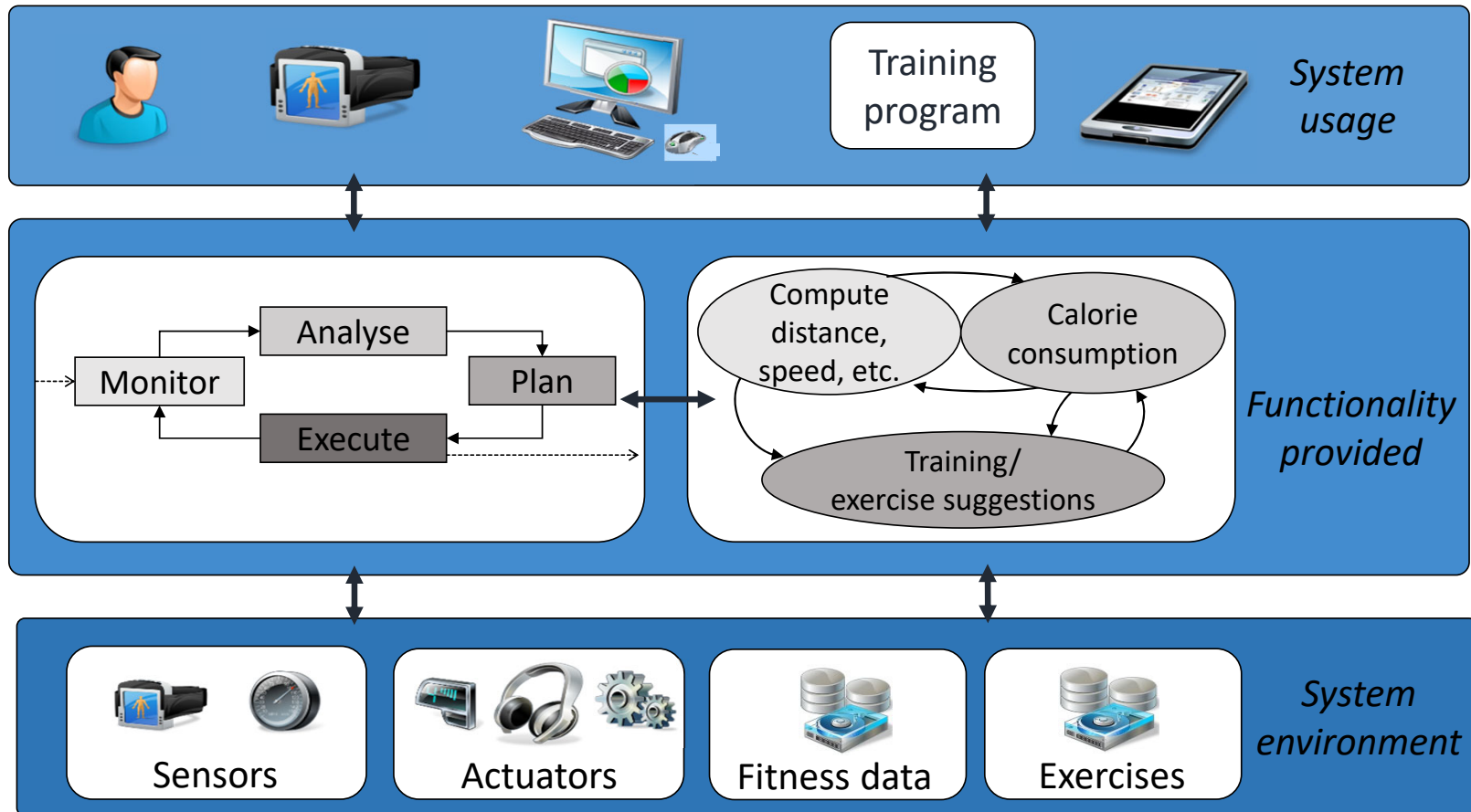
Order Management System



Thermal Comfort System

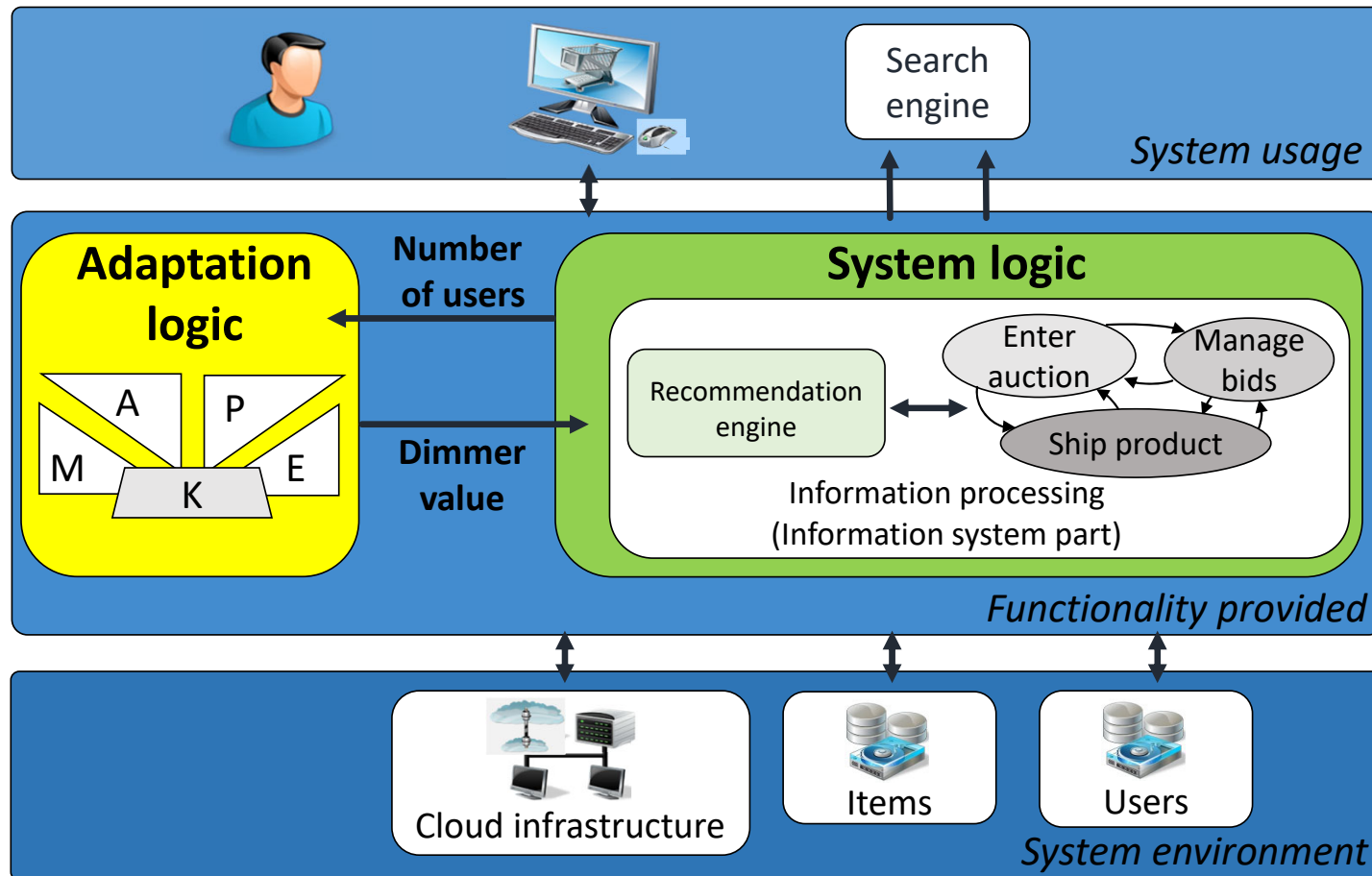


Software-Intensive Systems Fitness Tracking



Adaptive Systems

Adaptive Web Auction System



Evolution of Systems

- < Mid 90s: Differentiation between embedded and information systems: different requirements and development methods & models; different communities
 - > Mid 90s: “integration” of embedded and information system → software-intensive systems, merger of methods & models; object-oriented models
 - > 2010: Self-adaptive systems (software-intensive + adaptation logic with MAPE-K loop)
- Different characteristics and demands on requirements engineering
- Challenges for requirements engineering:
 - Increasing demand for innovative functionality and quality
 - Increasing demand for comprehensive context consideration, e.g., context monitoring, real-time identification of changes required, big data analytics, etc.
 - Adaptation of systems during run-time, ...

No Silver Bullet

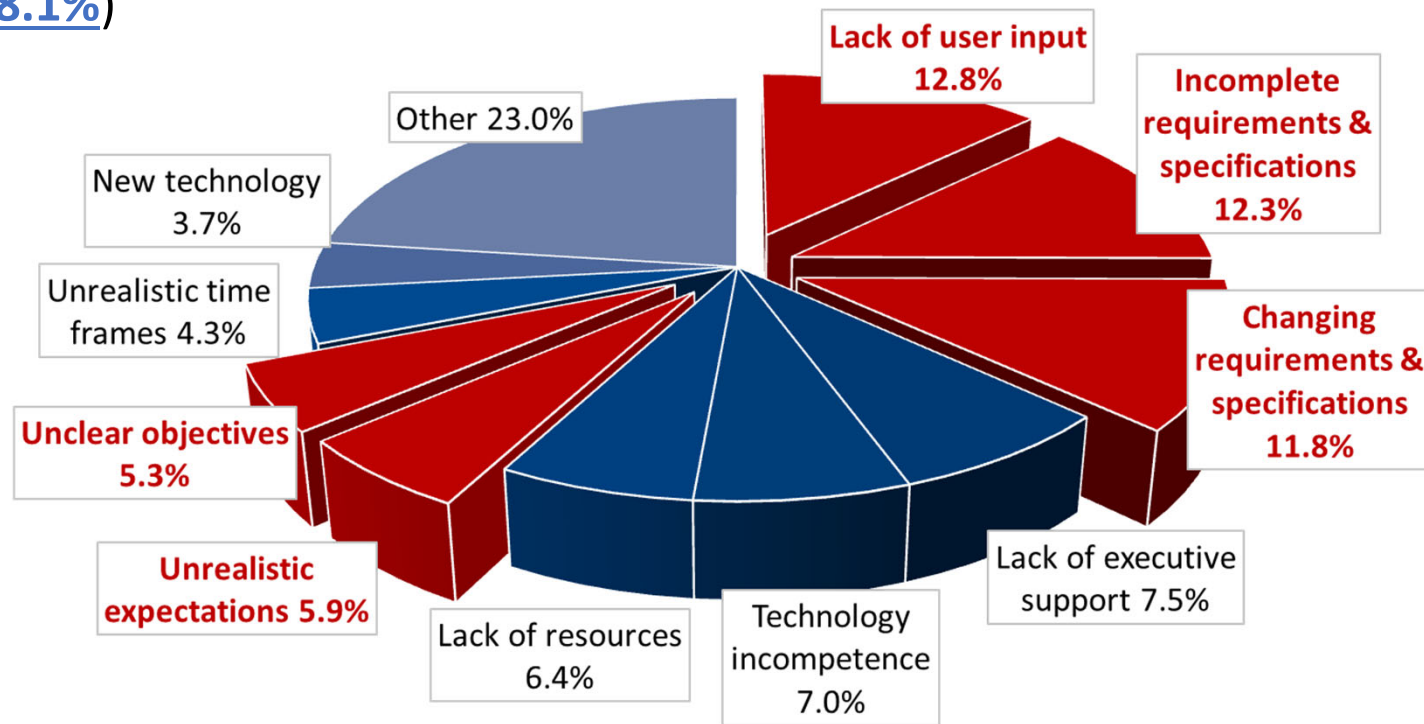
- “The hardest single part of building a software system is deciding precisely what to build.
- No other part of the conceptual work is as difficult as establishing the detailed technical requirements, including all the interfaces to people, to machines, and to other software systems.
- No other part of the work so cripples the resulting system if done wrong. No other part is more difficult to rectify later.”

CHAOS Report

- The CHAOS reports categorise projects into:
- Successful projects which delivered the software on time, within the budget constraints, and with all required functions.
- Challenged projects which were finished, but faced a resource overspend in budget and time, and/or delivered products with functional restrictions.
- Failed/cancelled projects which did not complete at all or delivered un-usable software.

CHAOS Report – Reasons Challenged Projects Resource Overspend and Functional Restrictions

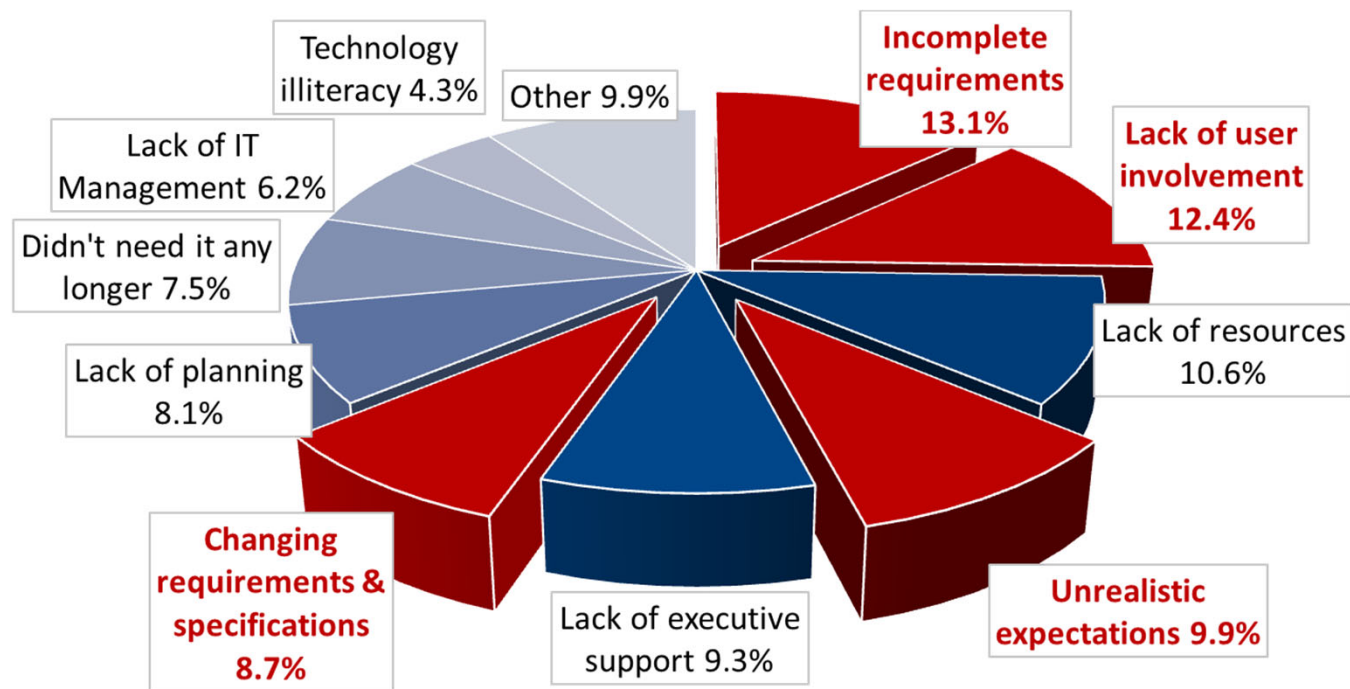
Reasons related to insufficient and poor requirements engineering!
(48.1%)



Based on [The Standish Group 1995]

CHAOS Report – Reasons Failed/Cancelled Projects

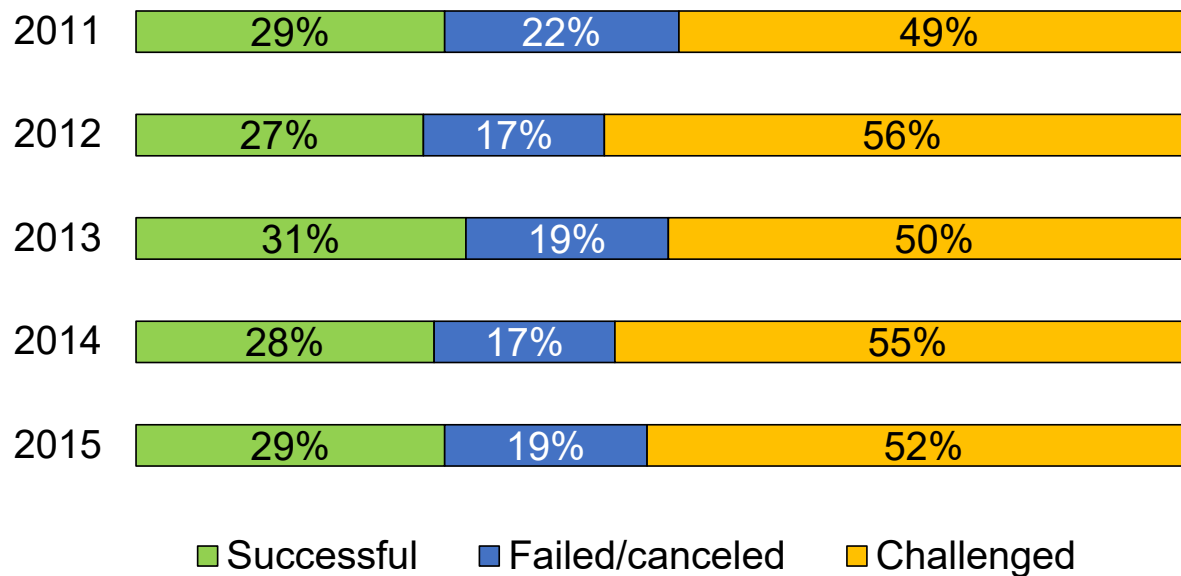
Reasons related to insufficient requirements engineering! (**44.1%**)



Based on [The Standish Group 1995]

CHAOS Report – Project Success and Failure

- The percentage of successfully finished projects did not increase over the years.



For very large projects,
only 6% of the projects
were successful in 2015.

Based on [The Standish Group 2015]

Effort for Correcting Requirements Defects

- Defect found after delivery
 - Small & non-critical projects:
increase of effort required for correction by factor 5!
 - Large & critical projects:
increase of effort required for correction by factor 100!

Therac-25 incident

- Computer controlled radiation therapy.
- Six accidents between 1985 – 1987 (overdose of radiation).
- Programming errors caused this to happen.
- Classic example of how software control of safety-critical systems can go bad if requirements are not done properly.
- The Therac-25 went into service in 1983. For several years and thousands of patients there were no problems. That is until June 3, 1985....



<https://hackaday.com/2015/10/26/killed-by-a-machine-the-therac-25/>



```
PATIENT NAME: John
TREATMENT MODE: FIX          BEAM TYPE: E      ENERGY (KeV):      10

                                ACTUAL          PRESCRIBED
UNIT RATE/MINUTE              0.000000          0.000000
MONITOR UNITS                  200.000000        200.000000
TIME (MIN)                      0.270000          0.270000

GANTRY ROTATION (DEG)          0.000000          0.000000      VERIFIED
COLLIMATOR ROTATION (DEG)      359.200000        359.200000      VERIFIED
COLLIMATOR X (CM)              14.200000        14.200000      VERIFIED
COLLIMATOR Y (CM)              27.200000        27.200000      VERIFIED
WEDGE NUMBER                    1.000000          1.000000      VERIFIED
ACCESSORY NUMBER                0.000000          0.000000      VERIFIED

DATE: 2012-04-16      SYSTEM: BEAM READY      OP.MODE: TREAT      AUTO
TIME: 11:48:58        TREAT: TREAT PAUSE      X-RAY              173777
OPR ID: 033-tfs3p     REASON: OPERATOR        COMMAND: █
```

Two modes of radiation treatment

- Electron beam
- Megavolt X-ray
- Field-light mode allowed patient to be correctly positioned by illuminating the treatment area with visible light.

Accidents

- The six documented accidents occurred when the high-current electron beam generated in X-ray mode was delivered directly to patients.
- Two software faults were to blame.
 - Op incorrectly selected X-ray mode before quickly changing to electron mode, which allowed the electron beam to be set for X-ray mode without the X-ray target being in place.
 - A second fault allowed the electron beam to activate during field-light mode, during which no beam scanner was active or target was in place.
- Previous models had hardware interlocks to prevent such faults, but the Therac-25 had removed them, depending instead on software checks for safety.
- The high-current electron beam struck the patients with approximately 100 times the intended dose of radiation, and over a narrower area, delivering a potentially lethal dose of radiation. "an intense electric shock". Several days later, radiation burns appeared through and through.

The Problem - Malfunction-54

- If the user selected X-ray mode, the machine would begin setting up the machine for high-powered X-rays. This process took about 8 seconds.
- If the user switched to Electron mode within those 8 seconds, the turntable would not switch over to the correct position, leaving the turntable in an unknown state.
- Pressing of P multiple times to proceed
- Due to the nature of this bug, conventional testing would never have identified the culprit. It took someone who was familiar with the machine – who worked with the data entry system every day, before the error was found.

The Software

- Atomic Energy of Canada Limited (AECL) never publicly released the source code, but several experts did obtain access for the investigation.
- The software appeared to have been written by a programmer with little experience coding for real-time systems. There were few comments, and no proof that any timing analysis had been performed.
- According to AECL, a single programmer had written the software based upon the Therac-6 and 20 code. However, this programmer no longer worked for the company, and could not be found.

Resources

- <https://ieeexplore.ieee.org/document/274940> (Therac-25 paper incident analysis)
- <https://en.wikipedia.org/wiki/Therac-25>
- <https://hackaday.com/2015/10/26/killed-by-a-machine-the-therac-25/>

Catastrophic Failures – Group Activity – 10 mins

- Select a catastrophic software failure to research. Examples include, Ariane 5 Rocket Explosion, Knight Capital Trading Glitch, Healthcare.gov Launch issues, Toyota Unintended Acceleration, Lastpass breach,.....

Investigate the following

- What happened?
- What were the consequences?
- What role did requirements engineering (or lack thereof) play?
- What could have been done differently?

Requirement

A requirement is either

- (1) a need or desire perceived by a stakeholder to solve a problem or to achieve an objective,
- (2) a capability or property that a system shall offer, or
- (3) a condition the system or its development shall satisfy.

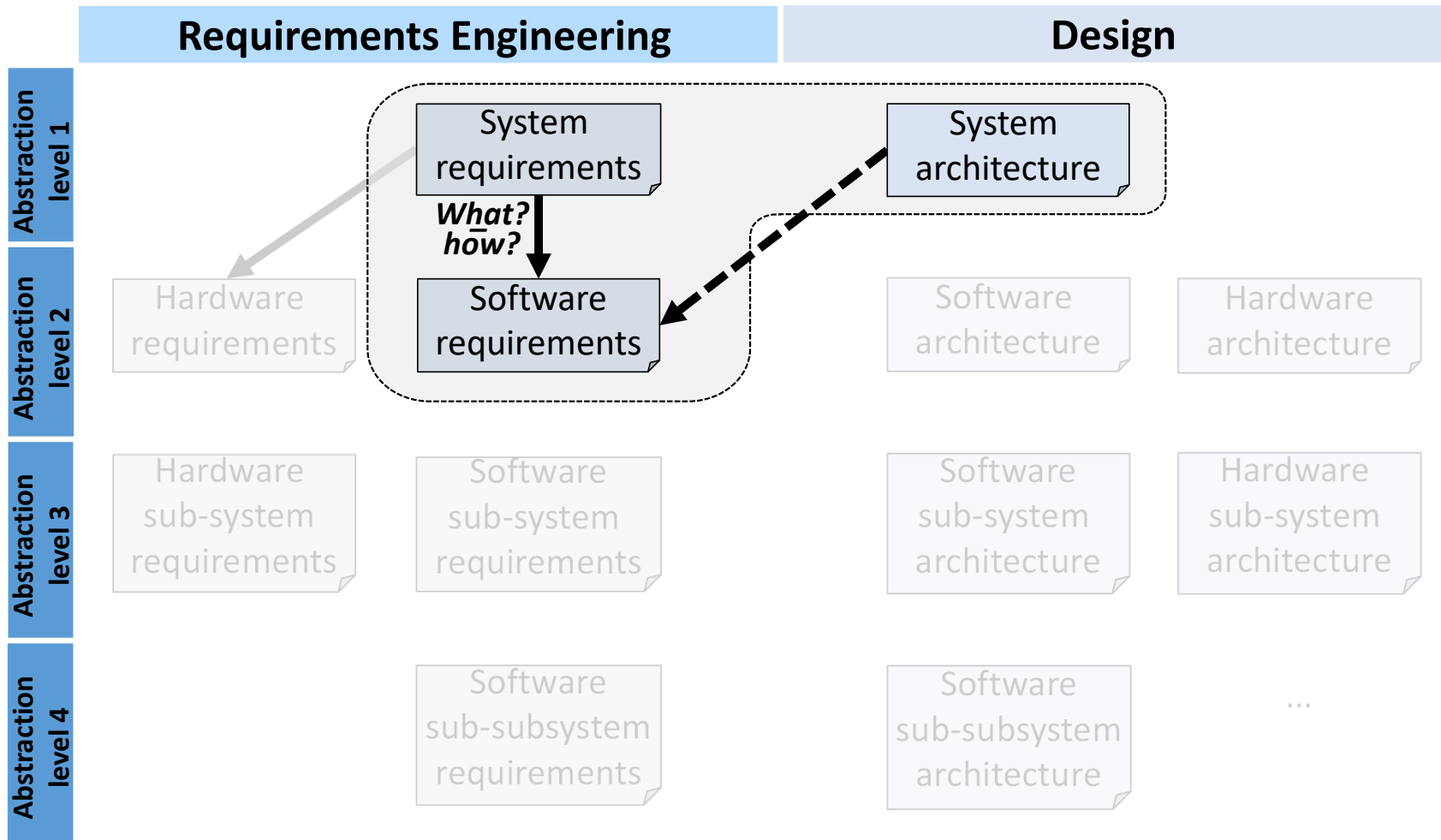
Requirements vs. Requirements Artefacts

Requirements Artefact

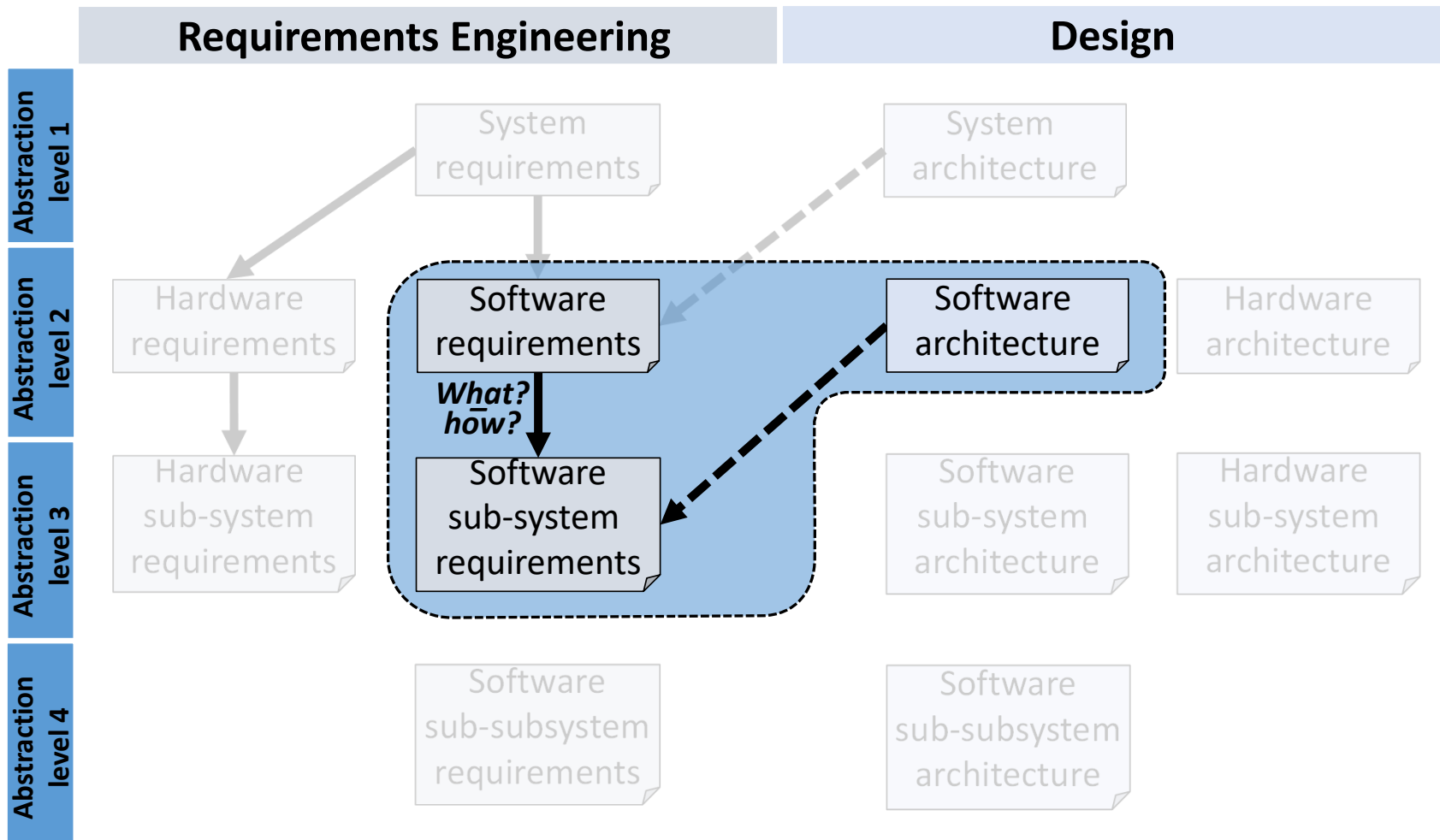
- The term “requirement” is often used as a homonym to refer to both:
 - (1) the requirement itself (e.g., condition or capability required by a stakeholder or other system), as well as
 - (2) the documented requirement
- ➔ Often unclear which of the two is meant
- To avoid ambiguity of the term “requirement” we distinguish between requirements and requirements artefacts

A requirements artefact is a documented requirement.

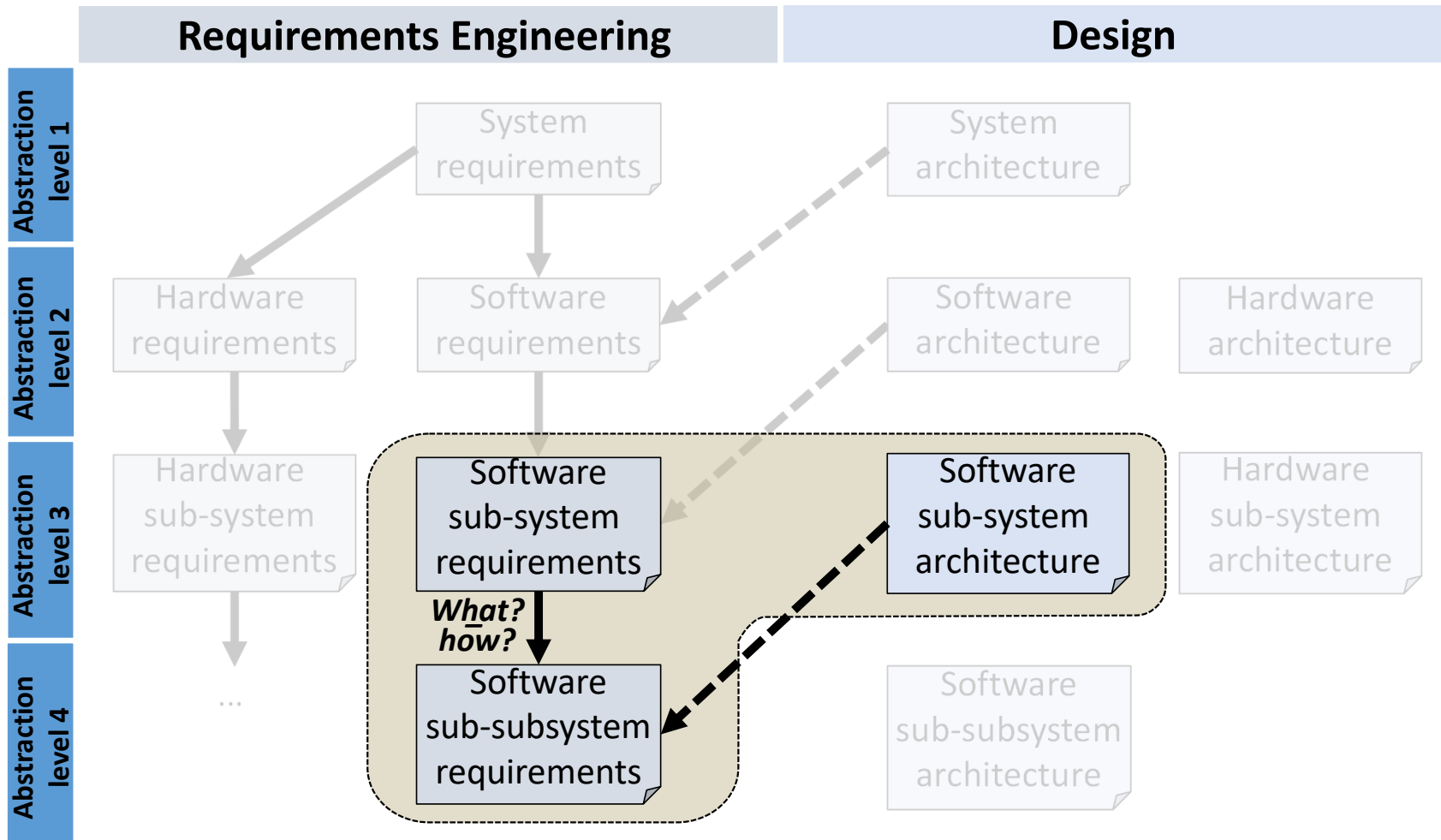
“What?” vs. “How?” – Within Requirements Engineering (1)



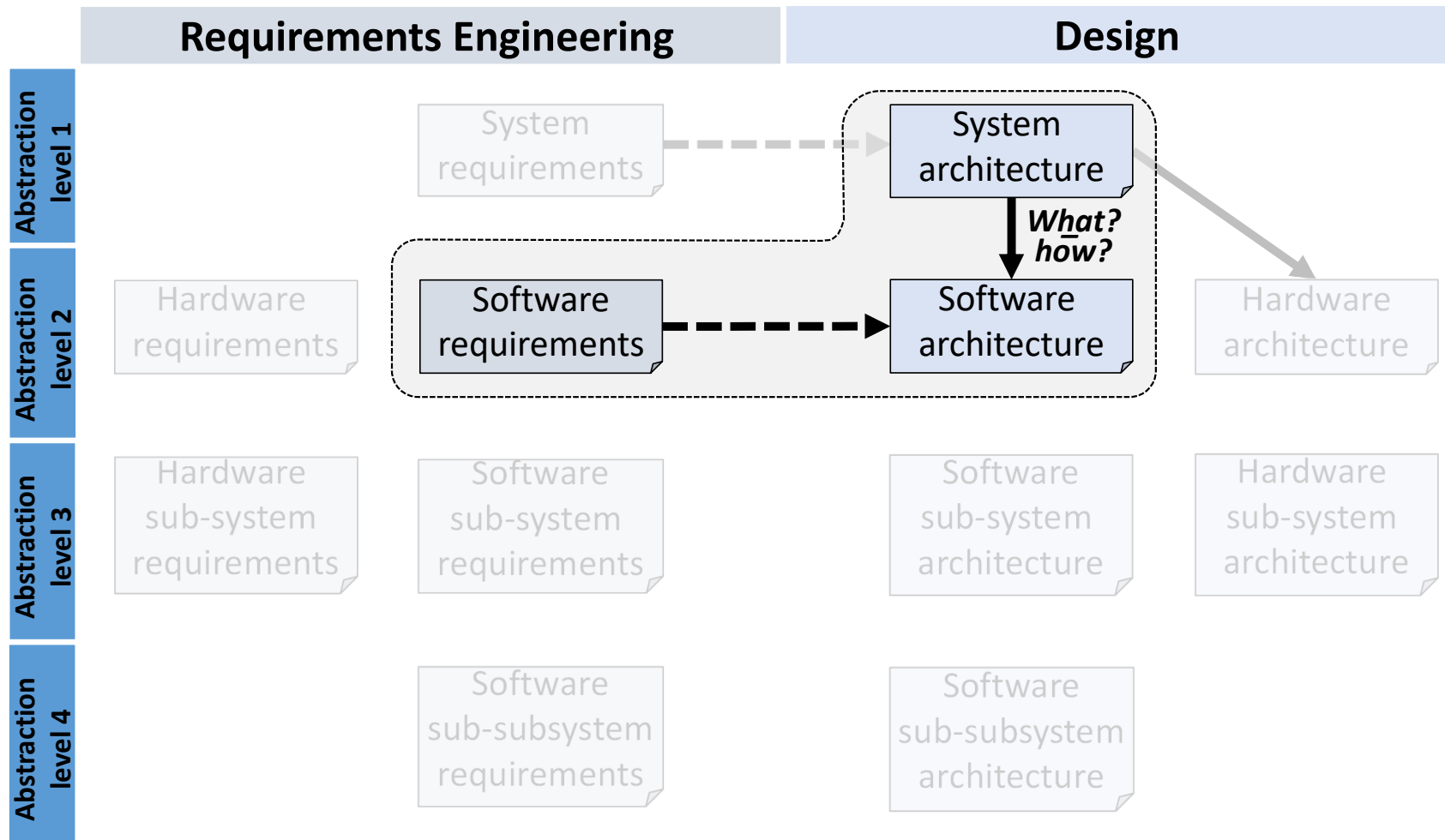
“What?” vs. “How?” – Within Requirements Engineering (2)



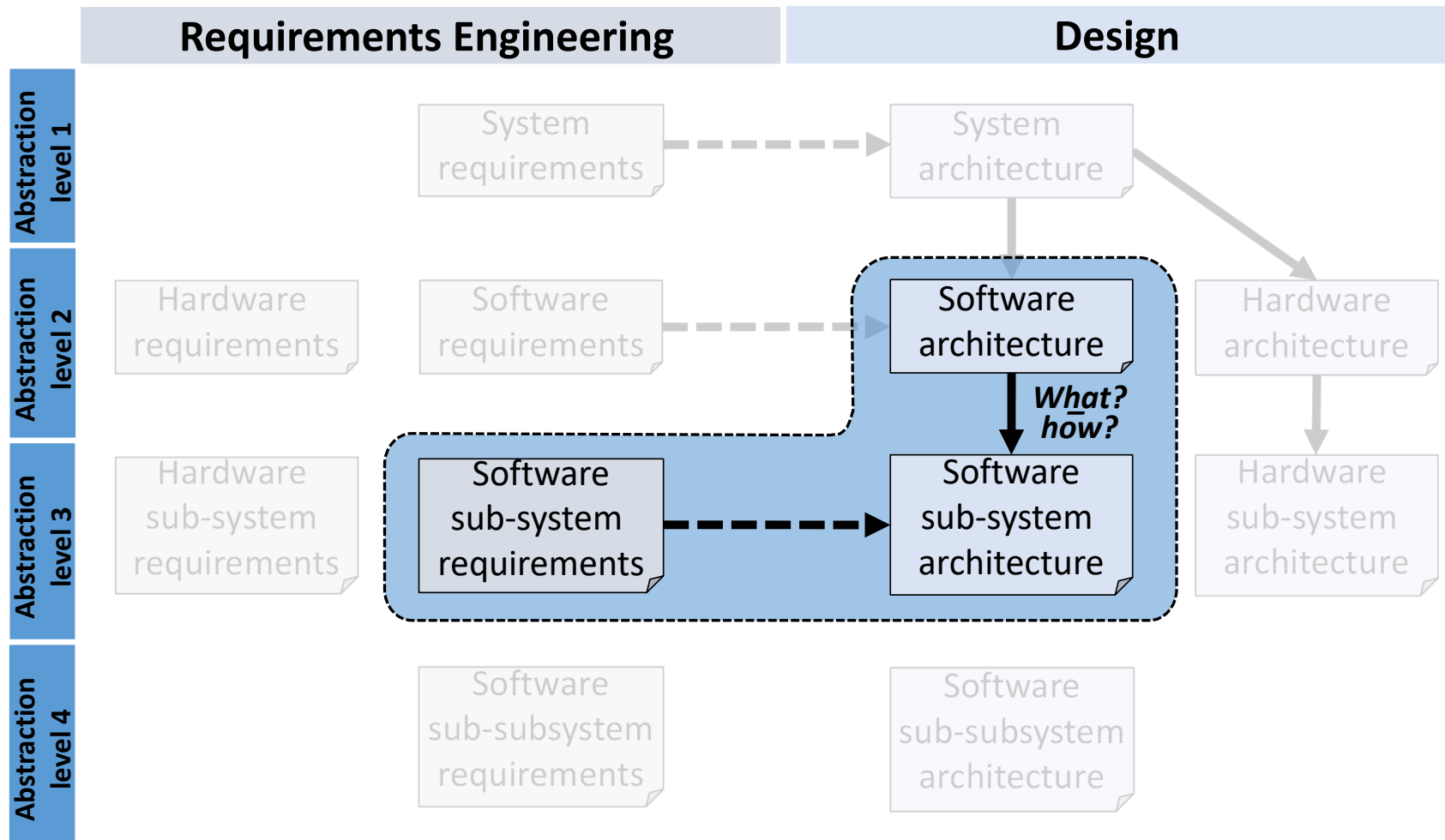
“What?” vs. “How?” – Within Requirements Engineering (3)



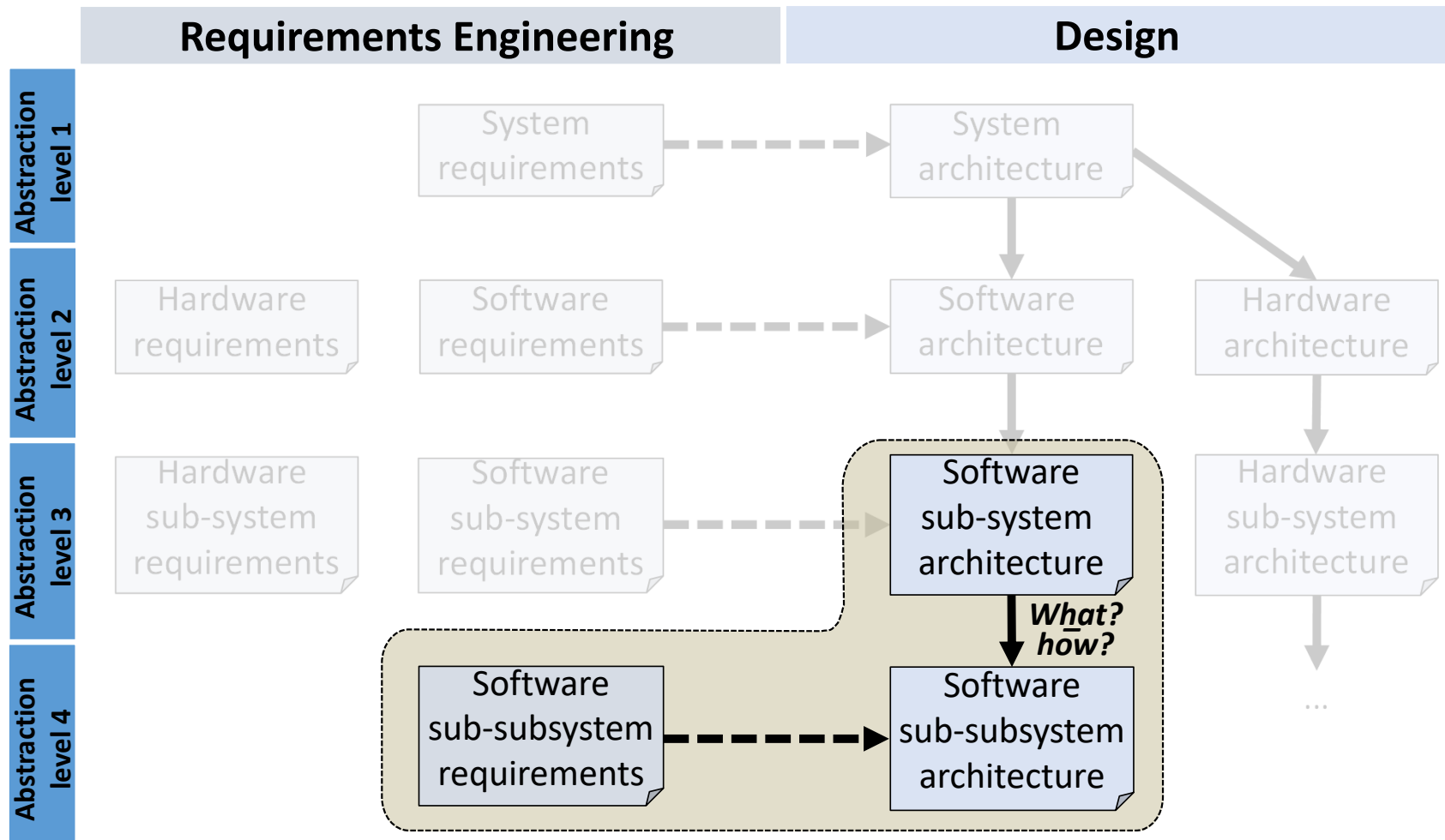
“What?” vs. “How?” – Within Design (1)



“What?” vs. “How?” – Within Design (2)



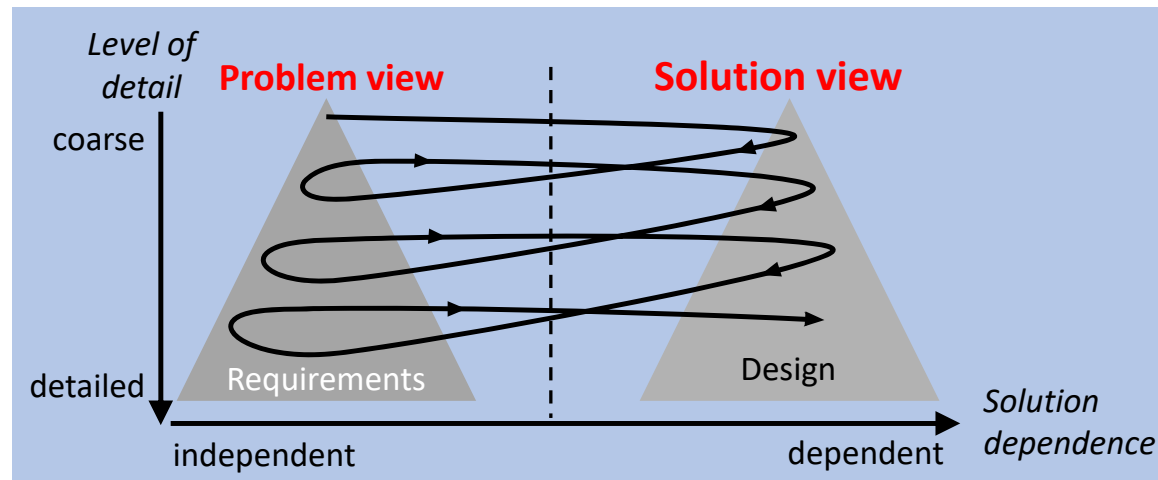
“What?” vs. “How?” – Within Design (3)



“What?” vs. “How?”

Co-Evolution of Requirements and Design

- Requirements define the problem (“what should be developed”)
- System design specifies the solution (“how the system should be developed”)
- Requirements are the basis for defining the architecture, but architectural decisions also influence the requirements
- Interactions between requirements and architecture:



Based on
[Ward and Mellor 1985, Nuseibeh 2001]

Two Types of Software Development Process Models

- Traditional

Software Development Process Models

- Waterfall Model
- V-Model
- Rational Unified Process

- Agile

Software Development Process Models

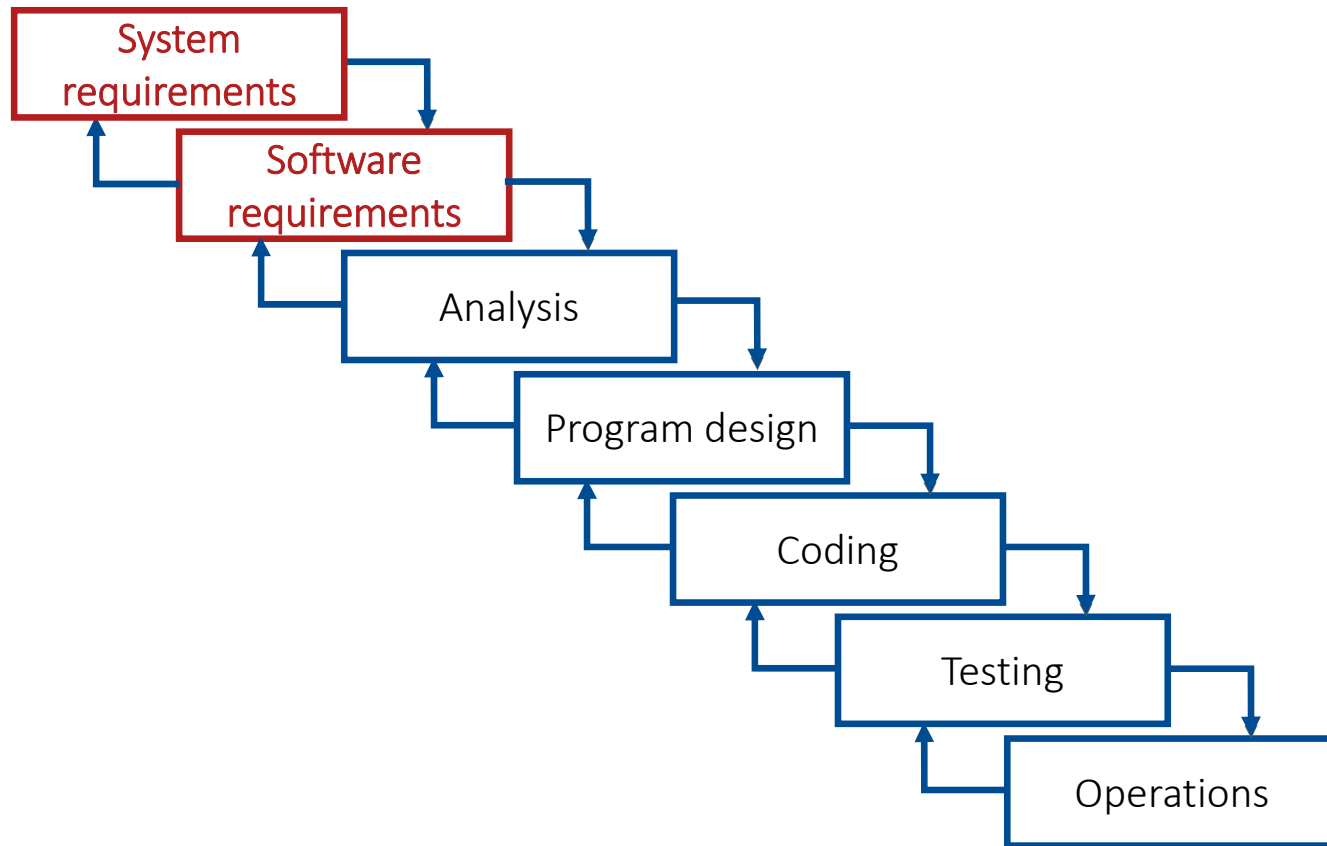
- Scrum
- Kanban



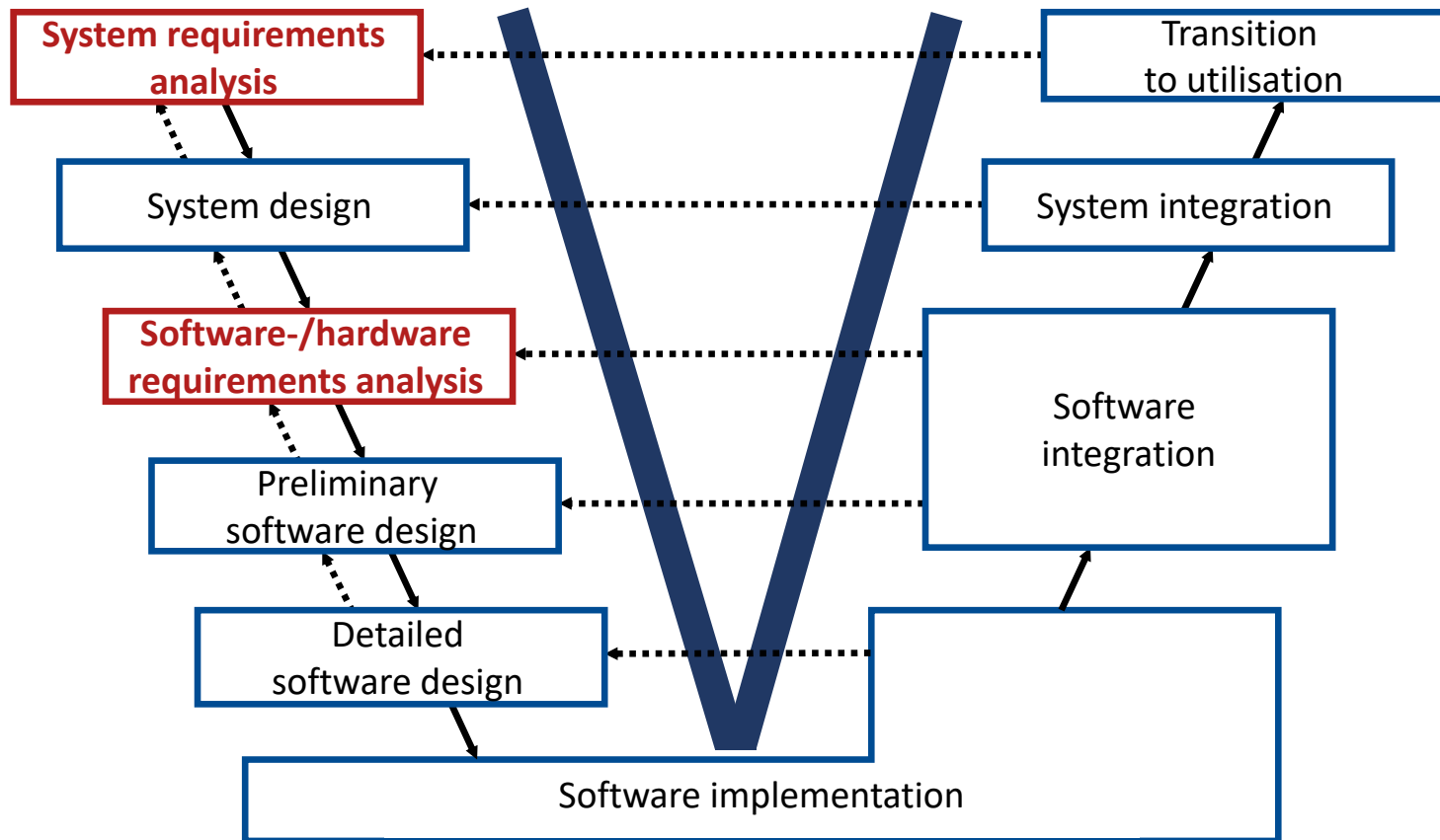
Are often **combined!**

Traditional Software Development Process Models – Waterfall Model

Process



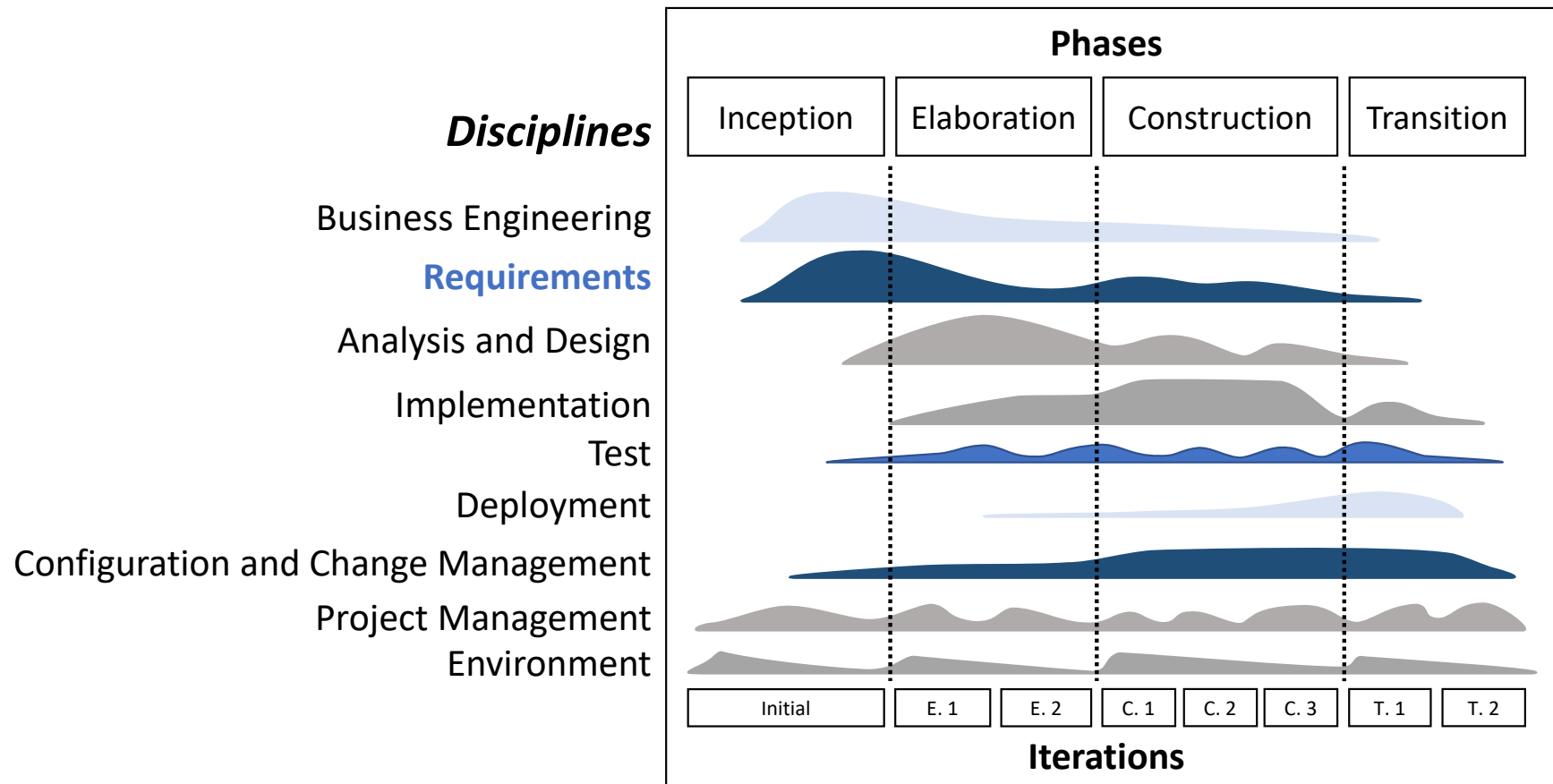
Traditional Software Development Process Models – V-Model Process



Based on [Dröschel and Wiemers 1999]

Traditional Software Development Process Models – Rational Unified Process

Process



Based on [Kruchten 2003]

Short Definitions

- Requirements Engineering is about discovering, analyzing, and communicating stakeholders' needs for a software system. (SOFT 468)
- Software architecture is about deciding and communicating how to structure the software system so as to facilitate the satisfaction of the stakeholders' needs (SOFT 466)

Long Definition

- Requirements engineering is a set of activities concerned with discovering, clarifying, analysing, communicating, and managing the evolution of
 - Stakeholders' needs for a software system
 - The context in which the software system will be used
 - The externally visible behaviors and quality properties the software must possess to satisfy the stakeholders' needs in a given context.
- It involves finding adequate tradeoffs between conflicting stakeholders' needs and the capabilities of software technologies.

Requirements Engineering Activities

- **Project Initiation** – short activity at the start of a project to define scope and goals
- **Elicitation** – discovering stakeholders' real needs
- **Decisions** – evaluate alternatives and decide what to build
- **Documenting Artefacts** – writing, reviewing and documenting precise specs of features to be implemented
- **Validation and verification** – check if requirements are correct
- **Management** – manage status and changes

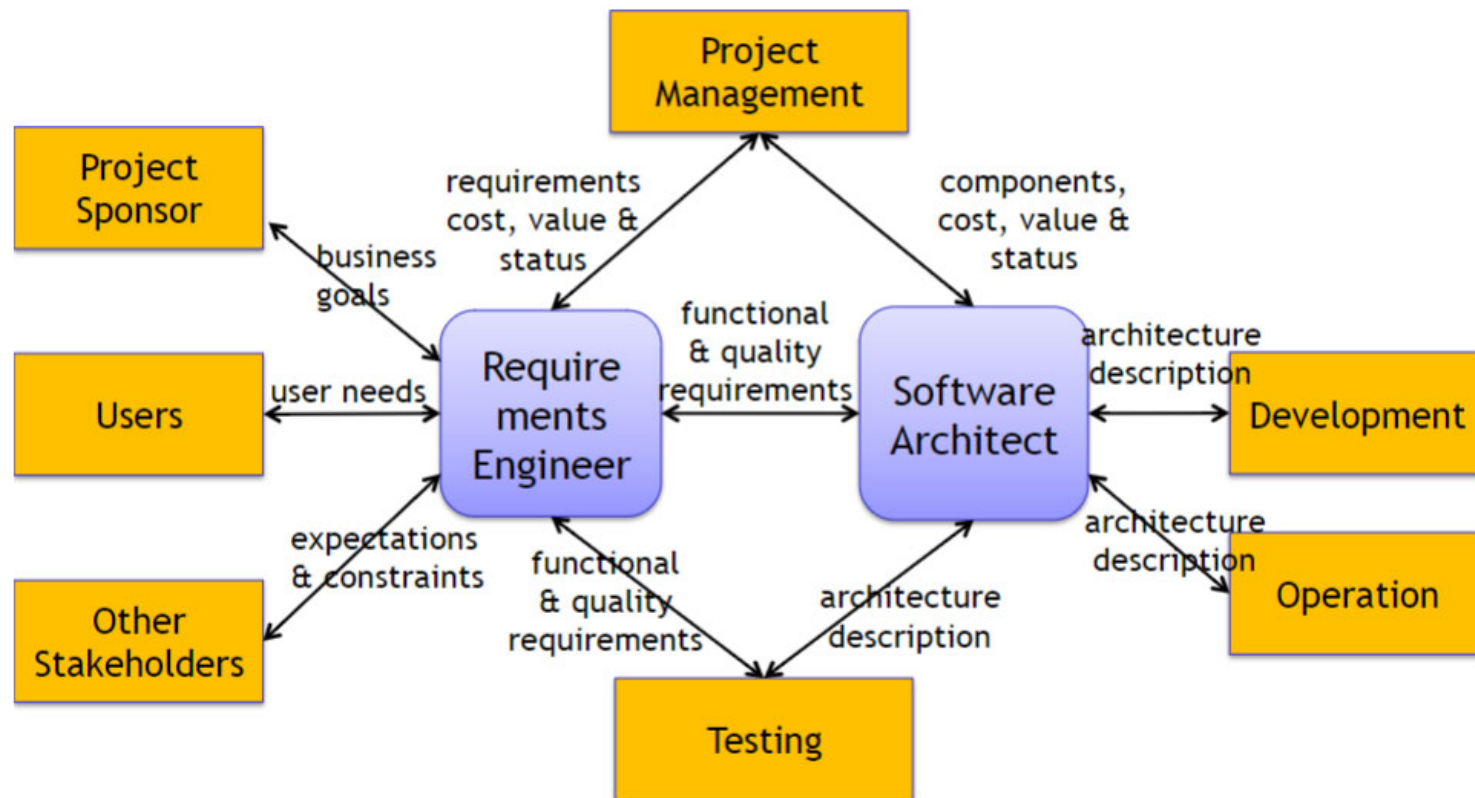
Some Misconceptions

- Requirements engineering is the first phase of development
 - RE is not a phase; it is a set of activities that occur throughout the SDLC (Software Development Life Cycle)
- The requirements engineer's role is to elicit client's requirements
 - RE is not just asking people what they want and writing it down.
 - RE is about analyzing the problems people face and identifying the best solutions to solve their problems – the best solutions are often not what they first asked for.

Why care about software architecture?

- Every system has an architecture, whether or not it is documented and understood.
- A **good architecture** makes it easier to
 - Satisfy the functional and quality requirements
 - Understand how the software works
 - Analyze the software properties
 - Test the software
 - Maintain and evolve the software
- A **bad architecture** makes all the above things much harder and sometimes impossible.

The Central Roles of Requirements Engineers and Software Architects in Software Development



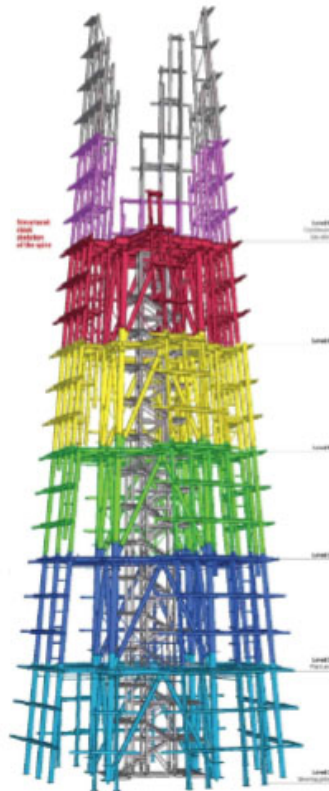
Modeling Requirements

- A model is a simplified description of a system or process to assist communication and reasoning



Modeling with multiple views

- Complex systems are best described using **multiple views**.



PIANTA PIANO NONO/ 9TH FLOOR PLAN
196 39580



Requirements models (a short list..)

- Goal model
 - Describes stakeholder's goals and refinements into subgoals, software requirements, constraints and domain assumptions.
- Conceptual model
 - Identifies and defines key concepts/entities of the application domain.
- Scenarios (Journey Maps)
 - Describes examples of system use.
- Context diagram
 - Describes the software system's inputs and outputs with its environment (people, hardware devices and other software systems)
- ...

The Expert – Video (~8 mins)

- Play video on computer. Will put link here after class.

In-class poll – not graded

- <https://polling.unl.edu/> (you can also use the Nebraska app)
- Join code: To be Announced

Summary

- RE is a critical success factor for software development.
- Requirements engineering activities
 - Are critical to satisfying stakeholders' needs
 - Play central roles in software development
 - Are important throughout development
 - Require modeling with multiple views (journey maps, user stories, sequence diagrams, context diagram, conceptual model, ...)

The goal of software development
is not to develop software



Jeff Patton
User Story Mapping, O'Reilly, 2014